

Kiste Phase 7 Spec

Data Repository + Dataset Intelligence

One-sentence spec: Kiste Phase 7 expands Kiste from code, image, and deployment awareness into dataset awareness by supporting Kaggle, Roboflow, Hugging Face Datasets, DVC, local folders, and cloud data sources as first-class workspace dependencies with metadata scanning, token access checks, dataset lineage, dataset-card generation, and export planning.

1. Phase 7 Position

Phase 7 comes after Kiste has repo scanning, remote providers, multi-repo workspaces, GitOps/SOPS, a Python library/CLI structure, and image/serverless planning. The new focus is data: datasets become first-class dependencies in the Kiste workspace graph.

Phase	Main focus
Phase 1	Local Docker repo scan + token inspection
Phase 2	GitHub/Hugging Face repo scan + CI testing
Phase 3	Multi-repo workspace graph + token matrix
Phase 4	GitOps repo + SOPS encrypted secrets
Phase 4.5	Review, refactor, architecture checkpoint
Phase 5	Python library + CLI
Phase 6	Many repo providers + Docker Hub + serverless planning
Phase 7	Many data repositories + dataset graph + dataset readiness

2. Why Phase 7 Matters

AI and data-driven systems are not only source code and containers. They also depend on training data, validation data, benchmarks, labeling platforms, dataset versions, and cloud storage. Without dataset awareness, Kiste cannot explain how a model was trained, whether a dataset can be accessed, whether the license is safe, or whether a future deployment can be reproduced.

- Training, validation, test, and benchmark datasets become workspace resources.
- Model repos can be connected back to the datasets that produced them.
- Dataset provider access can be checked before model training or evaluation fails.
- Dataset cards and lineage reports reduce hidden data risk.
- Export/download planning prevents accidental large downloads by default.

3. Data Providers to Support

Provider	Purpose in Kiste	Phase 7 behavior
Kaggle	Public/private datasets, competitions, notebooks ecosystem	Scan metadata, list files if available, check token, plan download

Provider	Purpose in Kiste	Phase 7 behavior
Roboflow	Computer vision datasets and annotation exports	Scan project/version, classes, splits, task type, export format
Hugging Face Datasets	ML dataset repos connected to model repos	Scan dataset repo files, card, formats, metadata
DVC	Git-connected data/model tracking with external remotes	Inspect .dvc files, dvc.yaml, remotes, tracked paths
Local folder	Small or development datasets	Profile files, formats, split folders, metadata files
S3 / GCS / Azure Blob	Cloud data lake or object storage sources	Metadata-only scan first, access check, no default bulk download

4. New Command Group

Phase 7 adds a new top-level command group:

```
kiste data
  add
  inspect
  scan
  list
  connect
  check
  profile
  lineage
  card
  export-plan
```

Example commands:

```
kiste data add kaggle owner/dataset-name --name traffic-dataset
kiste data add roboflow workspace/project/version --name weld-detection
kiste data add hf owner/dataset-name --name text-dataset
kiste data add dvc ./data.dvc --name training-data
kiste data add local ./datasets/weld-seams --name weld-seams

kiste data inspect weld-detection
kiste data scan
kiste data connect weld-detection training-repo --relation trains
kiste data connect weld-detection model --relation used-by-model
kiste workspace graph --include-data
kiste workspace check --include-data
```

5. DatasetSpec Model

Kiste should normalize dataset metadata into a DatasetSpec. Not every provider exposes all fields, so unknown values should be explicit rather than guessed.

```
{
  "name": "weld-detection",
  "provider": "roboflow",
  "dataset_ref": "workspace/project/1",
  "task_type": "object-detection",
  "format": "yolov8",
  "version": "1",
  "license": "unknown",
  "visibility": "private",
  "size": {
    "files": 1200,
    "bytes": null
  },
  "splits": {
    "train": 900,
    "valid": 200,
    "test": 100
  },
  "classes": ["weld", "seam", "defect"],
  "annotations": {
    "type": "bounding-box",
    "count": null
  },
  "quality": {
    "missing_labels": "unknown",
    "class_imbalance": "unknown",
    "duplicates": "unknown"
  },
  "security": {
    "contains_pii": "unknown",
    "license_risk": "unknown",
    "private": true
  }
}
```

```
}  
}
```

6. What Kiste Should Detect

Category	Fields to detect
Identity	Provider, dataset name, dataset ref, version, visibility
Task	Task type such as object detection, classification, text, tabular, audio, multimodal
Files	File formats, number of files, approximate size, folders, metadata files
Splits	Train, validation, test, benchmark split information
Labels	Classes, annotation type, label count, missing label hints
Security	Token requirement, private/public status, PII/sensitive-data warning
License	License metadata, unknown license warning, commercial-use risk warning
Export	Available export/download formats and target output plan
Lineage	Which training repo/model repo uses this dataset

7. Provider-Specific Detection

Roboflow

- Project type and version number.
- Export format, such as YOLOv8, COCO, VOC, classification folder, or segmentation format.
- Classes, train/valid/test splits, preprocessing, augmentation, and project metadata.
- Token permission for metadata read and export read.

Kaggle

- Dataset owner/ref and metadata.
- Dataset files and available download information.
- License if available, or warning when missing.
- Distinguish dataset from competition assets when possible.

Hugging Face Datasets

- Dataset card and repository files.
- Data file formats such as CSV, JSONL, Parquet, images, audio, or custom scripts.
- Repo privacy/access and dataset version/ref.
- Connection to model repos and training repos.

DVC

- .dvc files, dvc.yaml, dvc.lock, tracked data paths.
- Remote storage config without exposing secrets.
- Whether data can be pulled with available credentials.
- External dependency and output lineage from pipelines.

8. Dataset Graph

Phase 7 adds a dataset graph layer to the existing workspace, repo, image, and deployment target graph.

```
[Kaggle dataset] ---- v
[Roboflow dataset] -> [training repo] -> [model repo] -> [api image]
                    ^
[HF dataset] -----/
```

Example graph edges:

```
training-repo -> kaggle:traffic-dataset
training-repo -> roboflow:weld-detection
model-repo -> hf-dataset:text-dataset
api -> model-repo
worker -> model-repo
```

This allows Kiste to answer which datasets trained a model, which token can access each dataset, whether a model is reproducible, and whether any license/security risks are unknown.

9. Token System for Data Providers

Data providers use the same token metadata pattern as earlier Kiste phases: raw credentials should come from environment variables or future secret backends, not from plaintext workspace files.

```
kiste token add kaggle --name kaggle-dev --scope dataset:read --expires 2026-08-01
```

```
kiste token add roboflow --name roboflow-dev --scope dataset:read,export:read --expires 2026-08-01
```

```
kiste token add hf --name hf-data-dev --scope repo:read --expires 2026-08-01
```

tokens:

- name: kaggle-dev
 provider: kaggle
 scopes:
 - dataset:read
 expires_at: 2026-08-01
 secret_reference: env://KISTE_TOKEN_KAGGLE_DEV

- name: roboflow-dev
 provider: roboflow
 scopes:
 - dataset:read
 - export:read
 expires_at: 2026-08-01
 secret_reference: env://KISTE_TOKEN_ROBOFLOW_DEV

10. Dataset Readiness Check

Kiste should validate whether a dataset is usable before training, evaluation, or model reproduction depends on it.

```
kiste data check weld-detection

Dataset: weld-detection

[OK] Provider: Roboflow
[OK] Project type: object-detection
[OK] Version: 1
[OK] Splits: train/valid/test
[OK] Classes detected: 3
[OK] Export format available: yolov8
[WARN] License metadata missing
[WARN] PII/sensitive-data status unknown
[FAIL] Token cannot export dataset
```

Workspace-level data readiness:

```
kiste workspace check --include-data

Workspace data readiness:

[OK] training-repo -> weld-detection
[OK] model-repo -> hf-text-dataset
[WARN] Kaggle dataset license unknown
[FAIL] Roboflow token missing export permission
[WARN] Dataset lineage incomplete for model-repo
```

11. Dataset Card Generation

Kiste should generate a draft dataset card so data risks are documented instead of hidden inside code or notebooks.

```
kiste data card weld-detection --output docs/datasets/weld-detection.md
```

Section	Purpose
Dataset name/provider/version	Identify exact dataset source
Task type and file format	Explain intended ML use
Splits and classes	Document training/validation/test structure
License and visibility	Record commercial or sharing constraints
PII/sensitive-data notes	Surface privacy and compliance risk
Known limitations	Document imbalance, missing labels, sampling bias, or unknowns
Intended and not-intended use	Help avoid misuse
Training/model connections	Support reproducibility and lineage

12. Export Planning First, Download Later

Phase 7 should not automatically download huge datasets. The first version should inspect metadata, check access, and generate an export/download plan. Actual download should require an explicit future command.

```
kiste data export-plan weld-detection --format yolov8
```

```
Provider: Roboflow  
Dataset: weld-detection  
Version: 1  
Requested format: yolov8  
Estimated output: ZIP export  
Token needed: roboflow-dev  
Target path: data/roboflow/weld-detection/v1/  
Status: ready
```

Future explicit download command:

```
kiste data download weld-detection --confirm
```


13. Phase 7 Architecture

```
Kiste Python Library + CLI
|
v
Provider Registry
|-- Source Repo Providers
|-- Image Registry Providers
|-- Deployment Target Providers
|-- Data Providers
    |-- KaggleProvider
    |-- RoboflowProvider
    |-- HuggingFaceDatasetProvider
    |-- DvcProvider
    |-- LocalDatasetProvider
    |-- S3DatasetProvider
    |-- GcsDatasetProvider
    |-- AzureBlobDatasetProvider
|
v
Normalized Models
|-- RepoSpec
|-- ImageSpec
|-- DatasetSpec
|-- ModelSpec
|-- TargetPlan
|-- WorkspaceSpec
|
v
Workspace Graph
|
v
Readiness + Security + Reproducibility Report
```

DataProvider Interface

```
class DataProvider:
    def get_metadata(self, dataset_ref: str) -> dict:
        ...

    def list_files(self, dataset_ref: str) -> list[str]:
        ...

    def get_dataset_card(self, dataset_ref: str) -> str | None:
        ...

    def check_access(self, dataset_ref: str, action: str) -> bool:
        ...

    def export_plan(self, dataset_ref: str, format: str | None = None) -> dict:
        ...
```

14. What Phase 7 Should Not Do Yet

- No automatic large dataset download by default.
- No full data labeling workflow.
- No full data cleaning engine.
- No full PII detector.
- No full license legal engine.
- No automatic model retraining.
- No automatic cloud data lake provisioning.

- No paid dataset purchasing workflow.

Phase 7 should mainly inspect, lightly profile, check access, connect data to the workspace graph, generate dataset cards, plan export/download, and warn about license/security/reproducibility risks.

15. Acceptance Criteria

Phase 7 is complete when these commands work:

```
kiste data add kaggle owner/dataset --name traffic-data
kiste data add roboflow workspace/project/1 --name weld-detection
kiste data add hf owner/dataset --name hf-dataset
kiste data add dvc ./data.dvc --name training-data
kiste data inspect weld-detection
kiste data scan
kiste data connect weld-detection model --relation trains-model
kiste data check weld-detection
kiste data card weld-detection --output docs/datasets/weld-detection.md
kiste workspace graph --include-data
kiste workspace check --include-data
```

Required outputs:

- DatasetSpec model.
- Data-provider abstraction.
- Kaggle metadata scan.
- Roboflow dataset/version scan.
- Hugging Face dataset scan.
- DVC metadata scan.
- Dataset access/token check.
- Dataset graph connections.
- Dataset card draft.
- Export/download plan.
- Workspace data readiness report.

16. Suggested File Changes

```
src/kiste/
  data.py
  data_providers.py
  data_cards.py
  data_lineage.py
  models.py          # add DatasetSpec, DatasetAccessResult
  cli.py             # add kiste data commands as thin wrappers

tests/
  test_data_kaggle.py
  test_data_roboflow.py
  test_data_hf_dataset.py
  test_data_dvc.py
  test_data_cli.py
  fixtures/
    local_dataset_folder/
    dvc_project/

docs/
  phase7_data_repo_spec.md
  datasets/
```

17. References

- Kaggle API documentation: <https://github.com/Kaggle/kaggle-api/blob/main/docs/README.md>
- Roboflow export data documentation: <https://docs.roboflow.com/developer/export-data>
- Hugging Face Hub download guide: https://huggingface.co/docs/huggingface_hub/guides/download
- DVC data management documentation: <https://dvc.org/doc/user-guide/data-management>

18. Final One-Sentence Spec

Kiste Phase 7 expands Kiste from code, image, and deployment awareness into dataset awareness by supporting Kaggle, Roboflow, Hugging Face Datasets, DVC, local folders, and cloud data sources as first-class workspace dependencies with metadata scanning, token access checks, dataset lineage, dataset-card generation, and export planning.